Dynamic Programming Intro Review Sheet

The goal of this chapter isn't to memorize any single answer — it's to look at a problem and answer four questions first: what does dp[i] mean, how is it built from earlier answers, what's the base case, and which index do you return. Master those and every DP problem here becomes the same four steps.

Python 3 | Tutor style | Includes diagrams | Includes FAQ | Space-optimized versions

00 Review Order + The Four Questions

DP problems look wildly different but crack open the same way. Before writing any code, answer these four questions out loud — the code writes itself afterward.

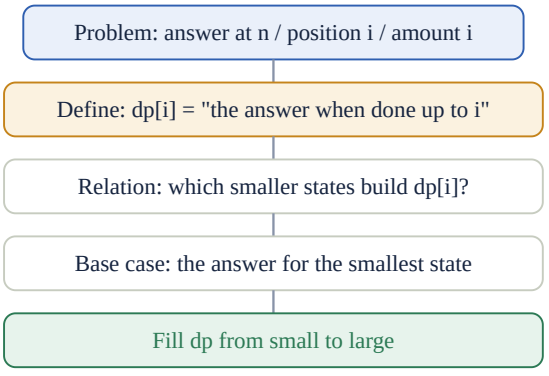
① What does	② How is dp[i] built	③ Base case + ④ which
`dp[i] mean?`	`from smaller states?`	`index to return?`

01 ← start	02	03	04	05	06
Fibonacci	**Climbing Stairs**	**Min Cost**	**House Robber**	**Coin Change**	**Space optimize**
smallest DP	count ways • +	position vs index • min	take / skip • max	two loops • inf	list → variables

When is a problem a DP problem? When a big problem breaks into **overlapping** smaller problems, and the answers to those small problems can be saved and reused. Fibonacci is the giveaway: computing `fib(5)` needs `fib(4)` and `fib(3)`; computing `fib(4)` needs `fib(3)` again. That repetition is the signal to store answers instead of recomputing them.

01 The DP Mindset

DP is not a fixed piece of code — it's a modeling procedure. The single most important step is always defining what `dp[i]` means, in plain English, **before** touching the keyboard.



The five modeling steps — same skeleton for every problem in this book

Universal template — memorize the shape, not any single problem `template`

```python
def solve_problem(data):
    # 1. handle the special inputs the problem allows
    if not data:
        return 0

    # 2. create dp; its size depends on what dp[i] means
    dp = [0] * (...)

    # 3. write the smallest state
    dp[0] = ...

    # 4. build from small problems up to large ones
    for i in range(...):
        dp[i] = ...

    # 5. return the target state
    return dp[...]
```

Translate dp[i] into English first. Every problem in this book starts with one sentence: "dp[i] = number of ways to reach step i" / "dp[i] = minimum cost to reach position i" / "dp[i] = most money robbable from the first i houses" / "dp[i] = fewest coins to make amount i." If you can't write that sentence, you're not ready to write the loop.

The most common transition bug

Only covering ONE choice ✗ wrong

```python
for i in range(2, len(nums) + 1):
    dp[i] = dp[i - 2] + nums[i - 1]   # only the "always take current" case
```

Comparing ALL allowed choices ✓ correct

```python
skip_current   = dp[i - 1]
choose_current = dp[i - 2] + nums[i - 1]
dp[i] = max(skip_current, choose_current)
```

Why the first one is wrong: it only writes the "definitely take the current element" case, and never compares against "skip the current element." A DP transition must cover **every** choice the problem permits — otherwise you're solving a more restrictive problem than the one asked.

02 Fibonacci · must master

The smallest possible DP: the current answer depends only on the previous two. Not a real business feature, but the prototype for many DP problems that follow.

`fib(0) = 0` `fib(1) = 1` `fib(n) = fib(n-1) + fib(n-2)` `0, 1, 1, 2, 3, 5, 8, ...`

Recursive version — for understanding only

fibonacci_recursive — slow for large n `recursion`

```python
def fibonacci_recursive(n):
    if n < 0:
        return None
    # Base case: the smallest problem has a direct answer.
    if n == 0:
        return 0
    if n == 1:
        return 1
    # Recursive case: break n into smaller problems.
    return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)
```

Why this is slow: it recomputes the same subproblems — `fib(5)` ends up calculating `fib(3)` multiple times. That wasted repetition is exactly what DP fixes by *saving* each answer once.

DP list version — write this one first

fibonacci_dp_list — dp list

```python
def fibonacci_dp_list(n):
    if n < 0:
        return None
    if n == 0:
        return 0

    # dp[i] = the i-th Fibonacci number.
    dp = [0, 1]

    for i in range(2, n + 1):
        dp.append(dp[i - 1] + dp[i - 2])

    return dp[n]
```

Watch the loop bound: the range goes to `n + 1` because `range` excludes its endpoint, and you must compute all the way through `dp[n]`. `dp[0]` is fib(0)=0, `dp[1]` is fib(1)=1. Edge cases: `n < 0` is undefined; when `n == 0` you must return before touching an assumed `dp[1]`.

Two-variable version — space optimization

fibonacci_variables — O(1) space — rolling

```python
def fibonacci_variables(n):
    if n < 0:
        return None
    if n == 0:
        return 0

    prev2 = 0  # fib(i - 2)
    prev1 = 1  # fib(i - 1)

    for i in range(2, n + 1):
        current = prev1 + prev2
        prev2 = prev1
        prev1 = current

    return prev1
```

Why you can drop the whole list: computing a new answer only reads the previous two values; older values are never used again. So keep two variables instead of an array. (More on this rolling pattern in § 07.)

Q A common off-by-one: `for i in range(2, n)`

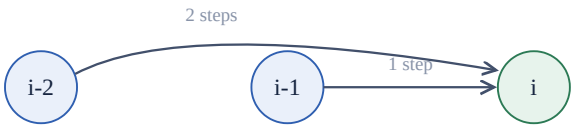
Writing `range(2, n)` only loops up to `n - 1`, so `dp[n]` never gets created and `return dp[n]` is an index error. It must be `range(2, n + 1)`.

◆ EXERCISES

1. Easy Write `fibonacci_dp_list(7)`, expected output `13`. Tests dp[i] and the loop bound.
2. Easy Convert the DP list version into the two-variable version. Tests "keep only the history you need."

03 Climbing Stairs · must master

Each move climbs 1 or 2 steps. How many distinct ways reach step `n`? The last step to reach `i` can only come from two places.



dp[i] = dp[i-1] + dp[i-2] — the + adds two independent valid routes

climb_stairs — count · +

```python
def climb_stairs(n):
    if n < 0:
        return 0

    # dp[i] = number of ways to reach step i.
    dp = [0] * (n + 1)
    dp[0] = 1

    if n >= 1:
        dp[1] = 1

    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]

    return dp[n]
```

Why dp[0] = 1? It represents "doing nothing" as one valid way. This lets step 1 derive naturally from step 0 — it's a modeling starting point, not a claim that you actually climbed a stair. The `+` is here because both sources are legitimate ways, so you add the counts.

Q Why not `dp[0] = 0, dp[1] = 1`?

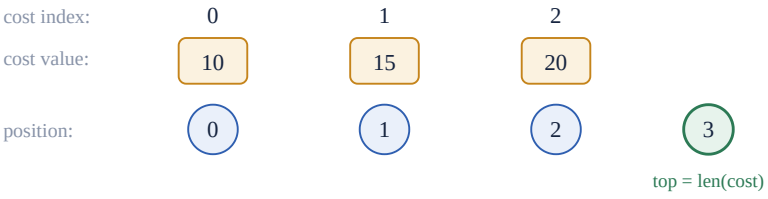
Setting `dp[0] = 0` makes "ways to reach step 0" become zero, which breaks the meaning of every later state — in this problem the empty path is a legitimate way. Keep `dp[0] = 1, dp[1] = 1`.

◆ EXERCISES

1. Easy `climb_stairs(4)` is `5`. Write out every dp entry by hand: `[1, 1, 2, 3, 5]`.
2. Medium Allow steps of 1, 2, or 3. Tests extending the transition to three sources.

04 Min Cost Climbing Stairs · must master · chapter focus

`cost[i]` is the price to step on stair `i`. Start from stair 0 or 1, move 1 or 2 each time. The crux: **dp[i] is a "position," not a cost index**. The top is at `len(cost)` — the position just past the last stair.



An extra dp slot (index 3) exists purely to represent the top

min_cost_climbing_stairs min

```python
def min_cost_climbing_stairs(cost):
    if not cost:
        return 0

    # dp[i] = minimum cost to REACH position i.
    # One extra slot to represent the top.
    dp = [0] * (len(cost) + 1)
    dp[0] = 0
    dp[1] = 0

    for i in range(2, len(cost) + 1):
        one_step = dp[i - 1] + cost[i - 1]
        two_step = dp[i - 2] + cost[i - 2]
        dp[i] = min(one_step, two_step)

    return dp[len(cost)]
```

Reading one_step: `dp[i-1]` is the min cost to first reach position `i-1`, and `cost[i-1]` is the price to step on stair `i-1`; added together, that's the total cost to take one step from `i-1` to position `i`. `two_step` is the same idea from `i-2`. Take `min` because the problem wants the cheapest.

Bug Comparison

WRONG	WHY IT'S WRONG	CORRECT
`two_step = dp[i-1] + cost[i-2]`	Coming from i-2, the prior min cost must be dp[i-2], not dp[i-1]	`dp[i-2] + cost[i-2]`
`return dp[len(cost) + 1]`	dp has length len(cost)+1, so the last valid index is len(cost)	`return dp[len(cost)]`

◆ EXERCISES

1. Easy Hand-trace `cost = [10, 15, 20]`, expected `15`. Tests why the top position is 3.
2. Medium Write the two-variable version, tracking prev2 / prev1 / current each round.
3. Easy `cost = []` should return `0`. Tests empty input.

05 House Robber · must master

A row of houses, `nums[i]` is the money in house `i`. You can't rob two adjacent houses. Maximize the take. Definition: `dp[i] = max money robbable from the first i houses` — so the current house's raw index is `i - 1`.

house_robber — the take/skip template max

```python
def house_robber(nums):
    if not nums:
        return 0

    # dp[i] = max money from the first i houses.
    dp = [0] * (len(nums) + 1)
    dp[0] = 0
    dp[1] = nums[0]

    for i in range(2, len(nums) + 1):
        skip_current = dp[i - 1]                # don't rob current
        rob_current  = dp[i - 2] + nums[i - 1]  # rob current
        dp[i] = max(skip_current, rob_current)

    return dp[len(nums)]
```

Trace: nums = [2, 1, 1, 2]

I	HOUSES CONSIDERED	SKIP CURRENT	ROB CURRENT	DP[I]
0	none	-	-	0
1	[2]	-	-	2
2	[2, 1]	2	1	2
3	[2, 1, 1]	2	3	3
4	[2, 1, 1, 2]	3	4	4

Answer is `4` — rob the first and last house.

> **Q Why can't House Robber just be `dp[i-2] + nums[i-1]` ?**
>
> That forces you to rob the current house. But sometimes the current house holds so little that skipping is better — e.g. in `[10, 1, 1]` you shouldn't rob the last house. Every house must compare **two** routes: `skip = dp[i-1]` vs `rob = dp[i-2] + nums[i-1]`, then take the `max`.

> **Q Two-variable init: `prev2 = 0, prev1 = nums[0]` — why this order?**
>
> prev2 corresponds to `dp[i-2]` and prev1 to `dp[i-1]`. At the start of the loop that's `dp[0]=0` and `dp[1]=nums[0]`. Writing them backwards (`prev2 = nums[0], prev1 = 0`) swaps the meaning of the states and corrupts every later calculation.

◆ EXERCISES

1. `Medium` `house_robber([2, 7, 9, 3, 1])` is `12`. Tests the take/skip comparison.
2. `Easy` `house_robber([])` is `0`. Tests the empty list.
3. `Medium` Rewrite with two variables — no dp list.

06 Coin Change · recommended fluency

Given coin denominations and an `amount`, find the fewest coins to make it, or `-1` if impossible. Definition: `dp[i] = fewest coins to make amount i`. If the last coin placed is `coin`, you first had to make `i - coin`.

coin_change — two loops, inf for "unreachable" — min · inf

```python
def coin_change(coins, amount):
    if amount < 0:
        return -1

    # dp[i] = fewest coins to make amount i.
    # inf means "we don't yet know how to make i."
    dp = [float("inf")] * (amount + 1)
    dp[0] = 0

    for i in range(1, amount + 1):
        for coin in coins:
            if i - coin >= 0:
                dp[i] = min(dp[i], dp[i - coin] + 1)

    if dp[amount] == float("inf"):
        return -1
    return dp[amount]
```

> **The `+ 1` :** it's the single `coin` you just placed as the last one. `i` is the amount you're currently making, `coin` is the denomination you're trying as the last coin, and you must check `i - coin >= 0` first so you never use a coin bigger than the amount.

Q Why does amount 4 naturally become two 2s?

The code doesn't "see" two 2s — it reuses an already-solved subproblem. Once the loop computes `dp[2] = 1` (one 2-coin), computing `dp[4]` with coin 2 looks at `dp[4 - 2] + 1 = dp[2] + 1 = 2`. DP kept the best answer for 2 around, so it composes into two 2s for free. **That's the whole power of DP: earlier answers don't get thrown away.**

Bug Comparison

WRONG	WHY IT'S WRONG	CORRECT
`dp = [0] * (amount + 1)`	Every amount but dp[0] starts falsely as "makeable with 0 coins"	`[float("inf")]...; dp[0] = 0`
`dp[i] = dp[i - coin] + 1`	Different coins give multiple candidates; plain assignment overwrites a better one	`dp[i] = min(dp[i], dp[i-coin]+1)`

◆ EXERCISES

1. Medium `coin_change([1, 2, 5], 11)` is `3` (5 + 5 + 1).
2. Easy `coin_change([2], 3)` is `-1`. Tests the unreachable state.
3. Medium Hand-trace the dp list for `coins = [1, 2], amount = 4`.

07 DP list → variables (rolling window)

When computing `dp[i]` only needs the last couple of states (like `dp[i-1]` and `dp[i-2]`), you can drop the whole array and keep just two rolling variables.

The name mapping — nothing new, just renamed slots — mapping

```
# dp list:        dp[i - 2]     dp[i - 1]     dp[i]
# variables:      prev2         prev1         current

current = ...              # compute new answer from OLD prev2 and prev1
prev2 = prev1              # slide the window one slot to the right
prev1 = current
```

VALUE IN DP LIST	NAME IN VARIABLES
`dp[i - 2]`	`prev2`
`dp[i - 1]`	`prev1`
`dp[i]`	`current`

Order matters — compute first, then slide. You must calculate `current` while `prev1` / `prev2` still hold the old values. If you write `prev1 = current` before computing, you destroy the history the next step needs. And a piece of advice: get the list version working **correctly** first, then optimize — readability always beats saving one array.

08 Bug + FAQ Table

When do you use + vs min vs max?

WHAT THE PROBLEM ASKS	OPERATION	EXAMPLE
How many ways	`+`	Climbing Stairs
Lowest cost / fewest count	`min`	Min Cost, Coin Change
Maximum gain	`max`	House Robber

Recurring gotchas

QUESTION	ANSWER IN ONE LINE
Why is dp length often `len(data)+1`?	dp[i] means "first i elements" / "reach position i"; the extra dp[0] represents "nothing chosen yet" / the start, so transitions need fewer special cases
Why return `dp[len(cost)]` not +1?	A list of length len(cost)+1 has last valid index len(cost); the top is defined as that position
When can a DP list become two variables?	When dp[i] only needs the last few states; write the list correctly first, then optimize
Why compute current before updating prev?	current still needs the old two values; overwrite them first and you lose the history
Why `float("inf")` in Coin Change?	You're taking a min; unreachable must be larger than any real answer so min() replaces it automatically

Back to the four questions to close the chapter: before writing any DP code, force yourself to answer — ① what does dp[i] mean, in one English sentence? ② which smaller states build it? ③ what's the base case? ④ which index do I return? Nail those four and DP stops being memorization and becomes muscle memory.

09 Review Path + Checklist

Work these in order. For each one, define dp[i] in English before writing the loop.

◆ REVIEW PATH

1. `Must` **Fibonacci** — write `dp[i] = dp[i-1] + dp[i-2]`, then the two-variable version.
2. `Must` **Climbing Stairs** — practice deriving "where the last step came from" from the problem text.
3. `Must` **Min Cost Climbing** — focus on the dp[i]-vs-raw-index relationship, and min.
4. `Must` **House Robber** — focus on the take/skip branches, and max.
5. `Fluency` **Coin Change** — focus on the double loop, unreachable states, and inf.
6. `Later` **2D DP, Knapsack, LCS, Edit Distance** — not yet; only once you can write the five above independently.

Chapter checklist

- `Define dp[i] in one English sentence`
- `+ for ways · min for cost · max for gain`
- `Base case avoids invalid indices`
- `Explain the +1 in range(2, n+1)`
- `Separate raw index i-1 from state i`
- `Length n+1 → last index is n`
- `float("inf") = "not yet reachable"`
- `Roll a 2-back DP into prev2/prev1`
- `House Robber writes both candidates`
- `Explain dp[4]=dp[2]+1 = two 2-coins`

Once every box is checked, this chapter isn't just "understood once" — you're ready to start building real DP muscle memory. Next up: Backtracking (subsets, combinations, permutations, generate parentheses), where recursion branches instead of rolling forward.